

Fundamentals of API Development

Breakout Lab 1.3: Peer Review & Interactive Docs

Day 2

Lab Overview

Goal: Review the Lab 1.2 solution spec against a completeness checklist, annotate the real controllers with springdoc-openapi, and compare the hand-written contract-first spec against the generated code-first version for the same two endpoints.

- **Format:** Pair work (review the spec together), then individual for annotation
- **Tools:** Swagger UI at `http://localhost:8080/swagger-ui.html`, IntelliJ, the quickstart API

NOTE

You just watched `DemoDocCompletenessChecker` score docs against a checklist and `DemoLiveDocsCheck` confirm the API serves `/api-docs` and Swagger UI. This lab applies both to the reference spec.

What's in the Companion Repo (1/2)

Open `day2/lab1_3_peer_review_and_interactive_docs/` in the companion repo.

Path	What it is
<code>start/PEER-REVIEW-CHECKLIST.md</code>	The checklist you fill in against the Lab 1.2 solution spec
<code>start/HealthController.annotated-template.java</code>	<code>HealthController.java</code> with springdoc annotation TODOs
<code>start/GreetingController.annotated-template.java</code>	Same pattern, for <code>GreetingController.java</code>
<code>solution/</code>	Filled-in checklist and fully annotated reference controllers

REFERENCE SPEC

Use the completed spec from Lab 1.2:

What's in the Companion Repo (2/2)

SETUP — GENERATING THE DOCS

```
cd ../api-dev-setup/quickstart-project
mvn spring-boot:run
# Then open http://localhost:8080/swagger-ui.html in a browser
# curl http://localhost:8080/api-docs          ← the raw JSON spec
# curl http://localhost:8080/api-docs.yaml    ← the YAML version
```

Step 1: Peer Review

Score the Reference Spec

Open `day2/lab1_2_writing_an_openapi_specification/solution/openapi-quickstart.yaml` from the companion repo. With a partner, fill in `start/PEER-REVIEW-CHECKLIST.md` against it:

- Does every endpoint have a `description` ?
- Does every schema field have a `description` and an `example` ?
- Is the response actually documented, or just assumed?

CAUTION

Docs that only show a 200 response are complete and dangerous at the same time. Write down anything you would fix, with a reason.

Step 2: Annotate the Real Controllers

From Template to Live Docs (1/2)

Open the real controller files in `api-dev-setup/quickstart-project/src/main/java/com/apidev/quickstart/controller/` :

1. `HealthController.java` , using the TODOs in `start/HealthController.annotated-template.java` as your guide, add `@Operation` , `@ApiResponse` , and `@Parameter` annotations.
2. `GreetingController.java` , same pattern using `start/GreetingController.annotated-template.java` .

From Template to Live Docs (2/2)

THEN REFRESH

Save the files. Restart the API so springdoc picks up the new annotations:

```
mvn spring-boot:run
```

Then refresh <http://localhost:8080/swagger-ui.html>. Your summaries and descriptions should now appear in the live console.

Verify from the command line:

```
curl -s http://localhost:8080/api-docs | python3 -m json.tool | head -40
```

Step 3: Compare the Two

Contract-First vs Code-First, Side by Side (1/2)

Save the generated spec to a file for side-by-side comparison:

```
curl http://localhost:8080/api-docs -o generated-openapi.json
```

Open the Lab 1.2 solution spec

(`day2/lab1_2_writing_an_openapi_specification/solution/openapi-quickstart.yaml`) and `generated-openapi.json` side by side.

Ask yourself:

- Does the hand-written spec's `description` match what you wrote in the annotation?
- Does the generated spec have anything the hand-written one is missing, or the reverse?

Contract-First vs Code-First, Side by Side (2/2)

STRETCH

Annotate a case the current controllers do not cover. Describe what `/swagger-ui.html` shows for a route that doesn't exist, and note whether it's worth documenting at all.

The Real Question

You now have one artifact built each way, contract-first (hand-written YAML) and code-first (generated from annotations), for the exact same two endpoints.

Which one would you trust to share with a consumer? Which one is easier to keep in sync?

BRING THIS TO THE DISCUSSION

Session 3's contract-first vs code-first discussion needs your evidence. You have both artifacts, be ready to defend which approach won.

Lab Completion Checklist

Checklist (1/2)

- [] Lab 1.2 solution spec reviewed against the completeness checklist
- [] At least one actionable fix written down with a reason
- [] `HealthController.java` annotated with `@Operation`, `@ApiResponse`, and `@Parameter`
- [] `GreetingController.java` annotated with the same pattern
- [] Swagger UI refreshed and showing your annotations live
- [] Hand-written spec compared against the generated `/api-docs` output
- [] At least one disagreement recorded between the two

Checklist (2/2)

DONE?

Keep both specs open. The contract-first vs code-first discussion is next, and you have the raw material for it.

Lab 1.3 Complete

Bring your annotated controllers and both specs to the discussion